

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Machina

Feature Guide

2026-10-03

Enterprise Linux hypervisor management platform.

Machina is a unified control plane for virtual machines running on libvirt / QEMU / KVM. It folds scattered virsh scripting, separate console gateways, and missing fleet observability into a single Rust daemon with a web dashboard, REST API, and the machinactl CLI. It is built for infrastructure teams and NOC operators who run VMs on their own metal and want cloud-grade lifecycle, consoles, security, and automation without vendor hypervisor lock-in.

100+ web console pages · **3** ways to drive it: Web, REST, CLI · **14** workspace crates, one shared core · built-in eBPF datapath

Version: October 2026 (updated 2026-10-03).

This is the customer-facing onboarding guide — how to access the product, your first workflows, and how to use every feature. A print-ready PDF of the same content sits alongside this file.

Contents

- 0. [Getting started — access & first workflows](#)
- 1. [VM Lifecycle & Compute](#)
- 2. [Storage & Disk Management](#)
- 3. [Networking](#)
- 4. [Consoles & Remote Access](#)
- 5. [Snapshots, Backup & Recovery](#)
- 6. [Fleet, HA & Multi-Host Control Plane](#)
- 7. [Observability & Operations](#)
- 8. [Security, Compliance & SOC](#)
- 9. [AI & Automation \(Zyra AI\)](#)
- | 0. [Applications, Templates & Provisioning](#)
- | 1. [Integrations & Migration](#)
- | 2. [Interfaces & Administration](#)

Getting started

How to access it

- **Web:** React dashboard at `https://<host>:5092` (self-signed TLS by default) — a classic single-host shell plus a multi-host platform shell under `/platform/*`.
- **CLI:** `machinactl` operates and deploys a host (`machinactl deploy` , `backup` , `health` , `doctor` , `audit verify`); `machina-daemon` is the server process.
- **API:** REST at base `/api/v1` with WebSocket streams under `/ws/v1` ; the live OpenAPI contract is served at `/api-docs` (e.g. `curl -sk https://<host>:5092/api/v1/health`).
- **Login:** Sign in on the web UI with a Linux host account via PAM — there is no separate Machina password; optional LDAP or OIDC SSO can front it.
- **Needs:** A Linux host with libvirt / QEMU / KVM and `/dev/kvm` (Machina builds and runs on Linux only).

Your first workflows

- **Deploy Machina onto a KVM host**
 1. Run `./machinactl doctor` to confirm `/dev/kvm` , `systemd`, `libvirtd`, and tooling are present.
 2. Run `./machinactl deploy` (deps → build → install → start → verify).
 3. Confirm the API is up: `curl -sk https://127.0.0.1:5092/api/v1/health` .
 4. Browse to `https://<host>:5092` and sign in with a Linux host account (PAM).
- **Create and boot your first VM**
 1. Web → VMs → New, or Web → Platform → Create (virt-install or domain XML).
 2. Pick an image/template, CPU, memory, disk, and network; optionally attach cloud-init.
 3. Submit to create — `POST /api/v1/vms` under the hood.
 4. Start it from Web → VMs → VM → Start (`POST /api/v1/vms/{name}/start`).
- **Open a browser console to a VM**
 1. Web → VMs → select the VM.
 2. Choose a console: VNC or SPICE for graphical, Serial/Terminal for text.
 3. The daemon proxies the session over an authenticated WebSocket — no client install.
 4. For Windows guests use the built-in RDP tab; download a `.vv` file to hand off to virt-viewer if preferred.
- **Lock down access**
 1. Populate `/var/lib/machina/roles.json` (an empty file makes every user admin) or map OIDC groups to roles.
 2. Set a strong `MACHINA_JWT_SECRET` and choose PAM, LDAP or OIDC.
 3. Replace the self-signed certificate with a CA-signed one.
- **Protect VMs with scheduled backups**
 1. Enable the daily timer: `./machinactl backup enable` .
 2. Point backups off-box (backup directory or NFS target) and set retention.
 3. Run one immediately: `./machinactl backup now` ; check with `./machinactl backup status` .
 4. Verify a backup's checksums via `POST /api/v1/backups/{id}/verify` .
- **Wire up Prometheus / OTLP observability**

1. Point your scraper at the native `/prometheus` endpoint on the host.
2. Or export metrics, logs, and traces over OTLP/HTTP to Grafana Alloy / an OTel Collector.
3. Watch live activity at Web → Events (SSE/WebSocket streams).
4. Configure alert rules and outbound webhooks at Web → Platform → Alert Rules / Webhooks.

- **Migrate a VM to KubeVirt**

1. Export the VM as a KubeVirt bundle: `GET /api/v1/vms/{name}/kubevirt-bundle` .
2. Apply it to a cluster: `POST /api/v1/vms/{name}/kubevirt/apply` .
3. Upload disks and start it: `POST /api/v1/vms/{name}/kubevirt/upload` then `.../kubevirt/start` .
4. Follow docs/kubevirt-migration.md for prechecks and the full path.

1. VM Lifecycle & Compute

Full create-to-delete control over every virtual machine, with live resource changes and zero-downtime moves.

- **Full VM lifecycle** — Create, start, stop, shutdown, reboot, pause, resume, clone, rename, and delete virtual machines from any surface. — *Every day-to-day VM operation in one place instead of ad-hoc virsh commands.*
 - **How:** Web → VMs → select VM → lifecycle actions; API `POST /api/v1/vms/{name}/start` (`/stop` , `/reboot` , `/pause` , `/resume` , `/clone`).
- **Dual creation backends** — Build VMs either through virt-install or from native libvirt domain XML, chosen globally or per request. — *Convenience for quick provisioning and full control when you need exact XML.*
 - **How:** Web → VMs → New (virt-install) or Web → Platform → Create (domain XML); API `POST /api/v1/vms` .
- **Golden image builder** — Build reusable base images asynchronously as background jobs using virt-builder or mkosi. — *Standardized, ready-to-clone images without hand-crafting each VM.*
 - **How:** Web → Platform → Content → build image (virt-builder / mkosi) as a background job.
- **Disk & NIC hot-plug** — Attach, detach, and resize disks and network cards on a running VM without a reboot. — *Adjust capacity live, avoiding downtime for storage and network changes.*
 - **How:** Web → VMs → VM → Hardware → attach/detach; API `POST /api/v1/vms/{name}/disk/attach` , `.../nic/attach` , `.../disk/resize/{target}` .
- **Live CPU & memory resize** — Change vCPU count and RAM allocation on running guests, with CPU pinning and scheduler tuning. — *Right-size workloads on the fly as demand shifts.*
 - **How:** Web → VMs → VM → resize; API `POST /api/v1/vms/{name}/vcpus/{count}` and `.../memory/{mb}` .
- **CPU & memory tuning** — Fine-grained cputune, memtune, per-vCPU pinning, and scheduler policy controls. — *Squeeze predictable performance out of shared hosts.*
 - **How:** Web → VMs → VM → CPU/Memory tuning (cputune / memtune / pinning).
- **Live migration** — Move running VMs between hosts with tunable max bandwidth and max downtime. — *Evacuate hosts for maintenance with no service interruption.*
 - **How:** Web → VMs → VM → Migrate, or Web → Platform → Migration; API `POST /api/v1/vms/{name}/migrate` .
- **Online block jobs** — Run block commit and pull operations, and abort jobs, on live disks. — *Consolidate or reshape disk chains without stopping the guest.*
 - **How:** Web → VMs → VM → Disks → block commit / pull / abort.
- **Guest agent by default** — Optionally inject the GuestKit QEMU guest agent into every new VM for cloud-init seeding and offline changes. — *In-guest coordination and reliable graceful shutdowns out of the box.*
 - **How:** Web → Settings → enable GuestKit guest-agent injection for new VMs.
- **Autostart & domain XML access** — Toggle per-VM autostart and read the raw libvirt domain XML for any machine. — *Bring VMs back after reboots and inspect exactly what libvirt runs.*
 - **How:** Web → VMs → VM → toggle Autostart / view XML; API `POST /api/v1/vms/{name}/autostart/{enabled}` , `GET .../xml` .

2. Storage & Disk Management

Manage libvirt pools, volumes, and VM disks, with an optional dedicated storage control plane.

- **Storage pools** — List, start, stop, refresh, and toggle autostart on libvirt storage pools. — *Keep backing storage online and current from the dashboard.*
 - **How:** Web → Storage → pools → start/stop/refresh/autostart; API `/api/v1/storage/pools` .
- **Volume management** — Create and delete storage volumes within any pool. — *Provision and reclaim VM disk space without touching the CLI.*
 - **How:** Web → Storage → select pool → Create / Delete volume.
- **Disk images view** — Browse disk images with per-VM disk resize and IO tuning. — *See and reshape what each VM actually consumes.*
 - **How:** Web → Disk Images → select image → resize / IO tuning.
- **Atlas storage integration** — Connect Atlas as a storage control plane spanning Ceph, NFS, and ZFS for VM disks, snapshots, and backups. — *Enterprise-grade shared storage under your fleet.*
 - **How:** Web → Platform → Atlas Storage → connect endpoint (see docs/atlas-storage.md).
- **Storage tiers** — Define and assign storage tiers so workloads land on the right class of disk. — *Match performance and cost to each workload automatically.*
 - **How:** Web → Platform → Storage Tiers → define/assign tiers.
- **Disk utility & ISO tools** — Platform disk utility plus ISO creation for boot and install media. — *Handle installer media and disk chores in the console.*
 - **How:** Web → Platform → ISO Create (and disk utility) to build boot/install media.

3. Networking

Virtual networks, host interfaces, filters, and a visual canvas for wiring the fleet together.

- **Virtual networks** — Create, delete, start, stop, autostart, and edit the XML of libvirt virtual networks. — *Full software-defined networking for guests without virsh.*
 - **How:** Web → Networks → create / start / stop / edit XML; API `/api/v1/networks` .
- **Host networking** — Manage host interfaces, bridges, routing tables, LLDP neighbors, and sysctl tuning. — *See and shape the physical network the VMs sit on.*
 - **How:** Web → Host Networking → interfaces / bridges / routes / sysctl.
- **Port forwarding & firewall** — Configure host port-forwarding rules and firewall entries. — *Expose guest services safely without external tooling.*
 - **How:** Web → Host Networking → port forwarding / firewall rules.
- **Network filters (nwfilter)** — Manage libvirt nwfilter rules that police guest traffic at the interface. — *Enforce per-VM traffic policy at the hypervisor edge.*
 - **How:** Web → NWFilters → create / edit nwfilter rules.
- **Network canvas & segments** — Visually design network segments and overlays across the fleet on an interactive canvas. — *Plan and understand multi-host networking at a glance.*
 - **How:** Web → Platform → Network Canvas → design segments/overlays.
- **Network overlay & sync** — Build overlay networks spanning hosts and keep network state synchronized across the fleet. — *Consistent connectivity for VMs wherever they run.*
 - **How:** Web → Platform → Networks → overlay / sync across hosts.
- **Native eBPF datapath** — `machina-bpfd` on every host loads Machina's own eBPF programs for per-VM flows, DNS and L7 visibility, byte accounting, packet capture and QoS, with no Cilium, Tetragon or other agent to install. — *Deep network visibility on every hypervisor out of the box.*
 - **How:** Web → Security Center → Native eBPF (22 tabs); API `/api/v1/bpf/*` ; fleet view `/api/v1/zeus-security/native-dataplane` .
- **Load balancing & DDoS shield** — XDP service load balancing (weighted, health-checked), QUIC connection-ID load balancing, and a per-source rate-limiting shield on the uplink. Fleet Cloud load balancers need no appliance VM. — *Front services and absorb floods at line rate on the host itself.*
 - **How:** Web → Native eBPF → Service LB / QUIC LB / Shield; Fleet Cloud → Load Balancers.
- **Kubernetes CNI** — `machina-cni` gives Kubernetes pods eBPF routing, NetworkPolicy (and compiled Cilium policies) and socket-level service load balancing. — *One datapath for VMs and containers.*
 - **How:** Install `machina-cni` on the node (see docs/ebpf/cni.md).

4. Consoles & Remote Access

Built-in VNC, SPICE, serial, SSH, and RDP proxies mean no separate console gateway.

- **noVNC graphical console** — Browser-based VNC console served directly by the daemon over WebSocket. — *Reach any VM's screen from a browser, no client install.*
 - **How:** Web → VMs → VM → Console → VNC (streamed over `/ws/v1`).
- **SPICE HTML5 console** — SPICE remote desktop rendered in the browser via a built-in HTML5 client. — *Rich desktop access with clipboard and multi-monitor support.*
 - **How:** Web → VMs → VM → Console → SPICE.
- **Serial & terminal streams** — Serial console and PTY terminal sessions streamed over authenticated WebSockets. — *Debug boot issues and drop into a shell from the UI.*
 - **How:** Web → VMs → VM → Console → Serial / Terminal.
- **SSH terminal proxy** — In-browser SSH terminal to VMs and ad-hoc hosts, with session TTLs. — *One authenticated path to shells across the fleet.*
 - **How:** Web → SSH (VM) or Web → Host SSH (ad-hoc host).
- **Native RDP for Windows guests** — A Windows guest is auto-detected, and RDP is exposed via a hypervisor NAT port-forward to the guest's port 3389 once the agent confirms Remote Desktop is actually listening. — *Windows desktops without a bolted-on gateway.*
 - **How:** Web → VMs → VM → Console → download the generated `.rdp` file and open it with Microsoft Remote Desktop (macOS) or `mstsc` (Windows).
- **ConsoleHub session broker** — Brokers console sessions with time-to-live limits, a cinema/wall multi-console view, and optional OIDC gating. — *Governed, auditable console access with a NOC-style live wall.*
 - **How:** Web → ConsoleHub (classic) or Web → Platform → ConsoleHub → cinema/wall view.
- **Virt-viewer handoff** — Download a `.vv` connection file to open a VM in a native virt-viewer client. — *Fall back to a desktop console client when preferred.*
 - **How:** Web → VMs → VM → download `.vv` and open in virt-viewer.

5. Snapshots, Backup & Recovery

Point-in-time snapshots plus scheduled, retention-managed backups and disaster-recovery workflows.

- **Per-VM snapshots** — Create, list, revert, and delete libvirt snapshots for any VM. — *Instant rollback points before risky changes.*
 - **How:** Web → Snapshots (or VM → Snapshots); API `POST /api/v1/vms/{name}/snapshots` .
- **Scheduled backups** — Daily backup timer exporting VMs into a backup directory or NFS target with configurable retention and optional disk inclusion. — *Automated, off-box protection with no manual steps.*
 - **How:** CLI `machinectl backup enable` ; schedule via API `POST /api/v1/backups/schedule` .
- **Backup lifecycle controls** — Enable, disable, run-now, and inspect backup status and logs from machinectl. — *Operate the whole backup pipeline from one command.*
 - **How:** CLI `machinectl backup now | disable | status | logs` .
- **Fleet snapshots & schedulers** — Controller-driven fleet-wide snapshot and backup scheduling across many hosts. — *Consistent protection policy for the entire estate.*
 - **How:** Web → Platform → Fleet Snapshots → define fleet snapshot/backup schedules.
- **Disaster recovery** — Dedicated DR workflows and backup targets for coordinated recovery. — *A rehearsed path back when a host or site is lost.*
 - **How:** Web → Platform → Backups → DR targets / restore; API `POST /api/v1/backups/restore` .

6. Fleet, HA & Multi-Host Control Plane

An optional enterprise controller adds HA failover, resource scheduling, and desired-state reconciliation across many hosts.

- **Fleet view** — Lightweight peer-based fleet status, metrics, alerts, and VM inventory in the daemon. — See *more than one host without the full controller*.
 - **How:** Web → Fleet → peer status / metrics / inventory.
- **Multi-host controller** — Optional control plane persisting state in embedded SQLite and driving hosts through per-host gRPC agents. — *Scale from one box to a managed estate*.
 - **How:** Deploy with `INSTALL_PLATFORM=1` (or deploy-remote `--platform`), then Web → Platform → Hosts.
- **High availability & fencing** — HA engine with health watchdog and fencing that restarts VMs elsewhere when a host fails. — *Workloads survive host failures automatically*.
 - **How:** Web → Platform → HA → configure fencing / anti-affinity (see docs/controller-ha.md).
- **DRS & rebalancing** — Distributed resource scheduling with placement, heatmaps, and fleet rebalancing. — *Keeps load spread evenly without manual juggling*.
 - **How:** Web → Platform → Placement → DRS recommendations / auto-balancing.
- **Smart placement** — Placement engine picks the best host for new or migrating VMs from live capacity. — *New VMs land where they fit, not where you guessed*.
 - **How:** Web → Platform → Placement → placement recommendations on create/migrate.
- **Desired-state reconciliation** — A reconcile engine continuously drives real state toward the declared intent. — *Drift self-heals instead of piling up*.
 - **How:** Runs automatically in the controller; observe drift/repair at Web → Platform → Tasks.
- **Task bus** — In-memory or NATS-backed task fan-out distributing work across the fleet. — *Reliable, scalable execution of fleet operations*.
 - **How:** Web → Platform → Tasks (list/cancel/retry); enable NATS fan-out in controller config.
- **Fleet power & maintenance** — Coordinated power control, maintenance missions, and host maintenance mode. — *Drain and service hosts safely on a schedule*.
 - **How:** Web → Platform → Maintenance → drain / maintenance mode / schedules.
- **Bare-metal enrollment** — Enroll and validate new hosts, with bare-metal automation hooks. — *Grow the fleet without hand-provisioning each node*.
 - **How:** Web → Platform → Enroll (or Bare Metal) → issue an enrollment token, run agent `install.sh` .

7. Observability & Operations

Prometheus metrics, history, OTLP export, alerts, events, and scheduled operations keep the fleet visible and automated.

- **Live & historical metrics** — Per-VM and host metrics with a metrics-history ring buffer persisted to disk. — *Trend performance over time, not just this instant.*
 - **How:** Web → Dashboard / Platform → Observability; API `/api/v1/metrics` .
- **Prometheus scrape** — A native `/prometheus` endpoint plus remote-write ingest for existing monitoring stacks. — *Drop Machina into your Grafana dashboards immediately.*
 - **How:** Point your scraper at `https://<host>:5092/prometheus` .
- **OTLP export** — Export metrics, logs, and traces over OTLP/HTTP to Grafana Alloy or an OpenTelemetry Collector. — *Feed one open pipeline instead of a bespoke agent.*
 - **How:** Web → Settings → configure OTLP/HTTP endpoint (see docs/guides/observability.md).
- **Alerts & webhooks** — Alert rules with an evaluator, notification channels, and outbound webhooks. — *Get told about problems where your team already looks.*
 - **How:** Web → Platform → Alert Rules and Web → Platform → Webhooks.
- **Event streaming** — Server-sent event and WebSocket streams of state changes and activity. — *Live dashboards and integrations react in real time.*
 - **How:** Web → Events; consume SSE/WebSocket streams under `/ws/v1` .
- **Health & doctor checks** — Deep health checks and host-readiness doctor with clear exit codes, plus a problems endpoint. — *Know a deploy is healthy before users hit it.*
 - **How:** CLI `machinactl health` and `machinactl doctor` ; API `GET /api/v1/health` .
- **PSI & cgroup pressure** — Surfaces Linux pressure-stall and cgroup signals for hosts. — *Spot resource contention before it becomes an outage.*
 - **How:** Web → Platform → Observability → PSI / cgroup pressure panels.
- **3D topology & datacenter view** — Interactive topology, datacenter, and infrastructure-graph views of hosts, VMs, and networks. — *Grasp the whole estate visually in seconds.*
 - **How:** Web → Platform → Topology / Datacenter view.
- **Scheduled jobs & operations** — Schedule recurring operations and VM schedules with a jobs runner. — *Automate routine ops on a calendar, hands-off.*
 - **How:** Web → Jobs (classic) or Web → Platform → Scheduled Jobs → add recurring job.
- **Reports & recommendations** — Generated reports plus rightsizing and optimization recommendations. — *Turn telemetry into concrete actions to take.*
 - **How:** Web → Platform → Reports and Web → Platform → Recommendations / Rightsizing.

8. Security, Compliance & SOC

A firewall control plane, network intelligence, SIEM/SOC tooling, and compliance frameworks harden the fleet.

- **Zeus firewall control plane** — Fleet-wide firewall with profiles, drift detection, lockdown, GitOps sync, approvals, and change checkpoints. — *Consistent, reviewable network policy across every host.*
 - **How:** Web → Platform → Zeus firewall → profiles / lockdown / approvals.
- **SOC & SIEM** — Security operations center with detection, event ingest, playbooks, and a SIEM pipeline. — *Detect and respond to threats without a separate SOC stack.*
 - **How:** Web → Platform → SOC → detections / event ingest / playbooks.
- **Threat hunting & attack surface** — Threat-hunting workspace and attack-surface-management (ASM) discovery. — *Proactively find exposure before attackers do.*
 - **How:** Web → Platform → Threat Hunting → run hunts / ASM discovery.
- **Compliance frameworks** — Compliance evaluation against frameworks with remediation guidance and exportable PDF reports. — *Audit-ready evidence generated from live state.*
 - **How:** Web → Platform → Security Center → Compliance → evaluate / export PDF.
- **Runtime enforcement & policy** — Policy engine plus runtime enforcement of security guardrails on running workloads. — *Rules that actually stop bad behavior, not just flag it.*
 - **How:** Web → Platform → Policy and Web → Platform → Runtime Enforcement.
- **eBPF enforcement with a safety lease** — Deny/allow policies, VM edge anti-spoofing, a QEMU sandbox, a BPF-LSM guard around QEMU and a node-isolation kill switch. Everything starts in observe/audit; enforce runs only under a lease that the kernel datapath itself expires, and nothing is persisted. — *Block real traffic without risking a permanent lockout.*
 - **How:** Web → Native eBPF → Overview (mode + lease), VM Edge, VMM guard, Node Isolation.
- **Guest per-container policy** — Per-container network allow rules and BPF-LSM restrictions (exec allowlist, W+X, devices, writable paths) enforced inside the guest by GuestKit, relayed through the QEMU guest agent. — *Least privilege for workloads inside your VMs, with no network path into the guest.*
 - **How:** Web → VMs → VM → Guest policy tab; API `/api/v1/vms/{name}/guest-policy` and `/guest-lsm`.
- **Signed audit log** — Tamper-evident audit log with optional line signing, syslog, and webhook shipping, verifiable via CLI. — *Prove who did what, and that the record wasn't altered.*
 - **How:** Web → Audit Log to review; verify integrity with CLI `machinectl audit verify`.
- **Linux audit integration** — Ingests Linux audit and SELinux AVC events with a health threshold. — *Host-level security signals folded into fleet health.*
 - **How:** Web → Audit Log → Linux audit / SELinux AVC events (ingested automatically).
- **Encrypted secrets & keys** — Secrets management plus AES-256-GCM encryption of stored provider keys with a master key. — *Credentials stay protected at rest, not in plaintext.*
 - **How:** Web → Secrets → store/manage secrets (provider keys encrypted with a master key).

9. AI & Automation (Zyra AI)

A controller-side AI engine adds natural-language ops, autonomous remediation, cost intelligence, and predictive SRE.

- **Natural-language ops** — Drive infrastructure with natural language through an intent router and NL-ops layer. — *Ask for what you want instead of memorizing APIs.*
 - **How:** Web → Platform → AI (Mission Control) → type an intent in natural language.
- **Autopilot & autonomous mode** — Autopilot and autonomous engines that plan and execute multi-step operations, gated by approvals. — *Routine remediation runs itself, with a human veto.*
 - **How:** Web → Platform → AI → Autopilot → enable, review/approve planned actions.
- **Incident commander & root cause** — Coordinates incident response with automated root-cause analysis and service-impact mapping. — *Faster diagnosis and a clear blast radius during outages.*
 - **How:** Web → Platform → Incident Commander → open incident / root-cause view.
- **Predictive SRE** — Predicts capacity and reliability issues and proposes SRE remediations before they bite. — *Fix tomorrow's problem today.*
 - **How:** Web → Platform → AI → Predictive SRE forecasts and proposed remediations.
- **Cost & FinOps intelligence** — Cost attribution, budgets, exposure FinOps, and rightsizing recommendations. — *Understand and cut infrastructure spend with data.*
 - **How:** Web → Platform → Reports / Rightsizing → cost attribution & budgets.
- **Migration readiness** — AI assessment of how ready a VM or workload is to migrate, with prechecks. — *Migrate with confidence, not surprises.*
 - **How:** Web → Platform → Migration → readiness assessment / prechecks.
- **Digital twin & infra memory** — Maintains a digital twin and long-term infrastructure memory of the estate. — *Decisions grounded in the real, remembered environment.*
 - **How:** Web → Platform → AI → digital twin / infrastructure memory view.
- **Knowledge & runbooks** — Searchable knowledge base, diagnosis, and generated runbooks for operations. — *Institutional know-how available on demand.*
 - **How:** Web → Platform → AI → Knowledge → search KB / generate runbook.
- **Agent marketplace** — Catalog of AI agents and pluggable LLM providers with configurable settings. — *Extend automation and choose your own model backend.*
 - **How:** Web → Platform → AI Providers / Marketplace → add agent or LLM provider.
- **AI VM builder** — Generate VM and blueprint definitions from high-level intent. — *Describe a machine and let the platform draft it.*
 - **How:** Web → Platform → VM Builder → describe intent, review generated spec.

10. Applications, Templates & Provisioning

A launchpad, marketplace, blueprints, and cloud-init studio turn raw VMs into repeatable, ready-to-run workloads.

- **Launchpad & spaces** — App launchpad organizing workloads into spaces with per-app detail views. — *Deploy and find applications like an app store, not a VM list.*
 - **How:** Web → Platform → Launchpad → browse spaces / open app detail.
- **Marketplace** — Built-in marketplace of deployable content and applications. — *One-click access to curated, ready workloads.*
 - **How:** Web → Platform → Marketplace → deploy an item.
- **Templates & catalog** — Template catalog with git-backed sources, image fetch, and readiness checks. — *Standardized starting points kept fresh automatically.*
 - **How:** Web → Platform → Templates → add git source / fetch image / deploy.
- **Blueprints** — Declarative blueprints describing multi-resource stacks to stamp out. — *Reproduce whole environments consistently.*
 - **How:** Web → Platform → Blueprints → author and stamp out a stack.
- **Cloud-init studio** — Author and manage cloud-init configuration to seed guests at first boot. — *Zero-touch first-boot customization for every VM.*
 - **How:** Web → Platform → Cloud-Init Studio → author cloud-init, attach on deploy.
- **Content library** — Central content store for images, ISOs, and provisioning artifacts. — *One shared source of truth for boot media.*
 - **How:** Web → Platform → Content → upload/manage images and ISOs.
- **GPU command center** — Discover, assign, and manage GPUs across the fleet. — *Put accelerators where AI and graphics workloads need them.*
 - **How:** Web → Platform → GPU Command Center → discover / assign GPUs.
- **USB & PCI passthrough** — Pass USB and PCI devices through to guests, gated by RBAC. — *Give VMs direct access to real hardware when required.*
 - **How:** Web → Devices → attach USB / PCI device to a VM (RBAC-gated).

11. Integrations & Migration

Move VMs to and from KubeVirt, and connect Machina to the wider Zyvor stack.

- **KubeVirt migration** — Export a VM as a KubeVirt YAML bundle and apply, upload, and start it on a Kubernetes cluster. — *A documented, repeatable path from libvirt to KubeVirt.*
 - **How:** API `GET /api/v1/vms/{name}/kubevirt-bundle` then `.../kubevirt/apply` , `/upload` , `/start` (docs/kubevirt-migration.md).
- **Fleet Cloud** — Native instance, flavor, network, and load-balancer management, backed entirely by Machina's own controller APIs. — *A full cloud-operator UI with no external cloud to wire up.*
 - **How:** Web → Fleet Cloud → instances / networks / images / load balancers.
- **Transiva / h2kvm** — Multi-cloud and cross-hypervisor VM migration into KVM. — *Bring VMs home from other platforms.*
 - **How:** Web → Import VM (Transiva / h2kvm) to migrate a source VM into KVM.
- **GuestKit offline assurance** — Offline VM migration assurance and guest inspection tooling. — *Validate guests before and after a move.*
 - **How:** Web → Import VM → GuestKit offline inspection / assurance.
- **Kubernetes & Kata** — View Kubernetes workloads and Kata containers alongside VMs, with KubeVirt VNC/console proxying. — *One console for both VMs and cluster workloads.*
 - **How:** Web → Kubernetes (Overview / Workloads) and Web → Kata Containers.
- **Vessel (Podman / Docker)** — List, create, and lifecycle local containers on the hypervisor host; Podman pod groups on `/containers/pods` . Live stats over WebSocket. — *Run sidecars and lab containers next to libvirt VMs without a separate tool.*
 - **How:** Web → Infrastructure → Containers; API `/api/v1/vessel/*` . Enable or point `[vessel].socket` in daemon config (on by default).
- **VMware & Proxmox awareness** — Controller APIs to interoperate with VMware and Proxmox sources. — *Onboard estates from other hypervisors.*
 - **How:** Web → Platform → Settings → Apps & Integrations → add VMware / Proxmox source.
- **Zyvor platform stack** — Plugs into Atlas (storage) and GuestKit (guest tooling); network enforcement and visibility are built in through native eBPF. — *Machina is the metal layer of a full private-cloud stack.*
 - **How:** CLI `machinactl integrations` shows KubeVirt/k8s/automation status; wire endpoints per integration.

12. Interfaces & Administration

Three ways to operate the platform, backed by PAM/LDAP/OIDC auth, RBAC, multi-tenancy, and one-command deploys.

- **Web dashboard** — React 19 Liquid Glass UI with classic single-host and platform multi-host shells covering 100+ pages. — *A polished, complete console for the whole platform.*
 - **How:** Browse `https://<host>:5092` (classic shell; multi-host under `/platform/*`).
- **REST + WebSocket API** — Complete `/api/v1` REST surface and `/ws/v1` streams with a published OpenAPI contract at `/api-docs`. — *Automate anything and generate typed clients.*
 - **How:** Call base `/api/v1` ; stream `/ws/v1` ; browse the contract at `/api-docs` .
- **machinactl CLI** — One CLI for deps, build, install, deploy, upgrade, backup, verify, health, doctor, audit, and integrations status. — *Stand up and operate a host with single commands.*
 - **How:** Run `machinactl <command>` (e.g. `machinactl deploy` , `machinactl health`).
- **PAM / LDAP / OIDC auth** — Log in with Linux host accounts via PAM, with optional LDAP and OIDC SSO and Active Directory integration. — *Use the identity system you already run.*
 - **How:** Web → Login signs in via PAM by default; enable LDAP/OIDC in the daemon config (see docs/ldap-auth.md).
- **RBAC roles** — Admin, Operator, and ReadOnly roles mapped from a roles file, OIDC groups, or scoped API tokens. — *Least-privilege access for every operator.*
 - **How:** Map roles in `/var/lib/machina/roles.json` or via OIDC groups; manage at Web → Platform → Users.
- **Scoped API tokens & sessions** — Issue prefixed, scope-limited API tokens and manage HttpOnly session cookies with admin revocation. — *Grant machines and people exactly the access they need.*
 - **How:** Web → Platform → API Keys to issue scoped tokens; Web → Admin Sessions to revoke sessions.
- **Projects & multi-tenancy** — Organize resources into projects with users, groups, and per-project scoping. — *Cleanly separate teams and tenants on shared infrastructure.*
 - **How:** Web → Platform → Projects → create project, assign users/groups.
- **Flexible deployment** — Single-host machinactl, remote rsync deploy, multi-host controller/agent, or a daemon-only Helm chart. — *Install the way that fits your environment.*
 - **How:** CLI `machinactl deploy` (single host) or `scripts/deploy-remote.sh USER@HOST` (add `--platform` for the controller and agent).
- **Support & upgrade tooling** — In-console support bundle, upgrade flows, and self-updating deploys with health verification. — *Maintain and troubleshoot the platform from inside it.*
 - **How:** Web → Platform → Support (bundle) / Upgrade; CLI `machinactl upgrade` .

Good to know: Machina builds and runs on Linux only (it depends on libvirt/QEMU/KVM headers and `/dev/kvm`) — the workspace does not compile on macOS, though the web UI alone does. HTTPS ships with a self-signed certificate that should be replaced with a CA-signed one for production. Security defaults matter: an empty `roles.json` grants every user admin, and the dev-only auth-bypass flags must never be set in production. The multi-host controller and agent tier is systemd-only (the Helm chart deploys just the daemon), and SAML is config-only today. Integrations such as KubeVirt and Atlas are disabled by default and require their own endpoints or credentials; Fleet Cloud is native and always on.

